

Отчёт по лабораторной работе 8

Программирование цикла. Обработка аргументов командной строки.

Вахиш Дуаа Иссам Али Ахмед

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Реализация циклов в NASM	6
2.2	Обработка аргументов командной строки	12
2.3	Задание для самостоятельной работы	17
3	Выводы	20

Список иллюстраций

2.1	Программа в файле lab8-1.asm	7
2.2	Запуск программы lab8-1.asm	8
2.3	Программа в файле lab8-1.asm	9
2.4	Запуск программы lab8-1.asm	10
2.5	Программа в файле lab8-1.asm	11
2.6	Запуск программы lab8-1.asm	12
2.7	Программа в файле lab8-2.asm	13
2.8	Запуск программы lab8-2.asm	13
2.9	Программа в файле lab8-3.asm	14
2.10	Запуск программы lab8-3.asm	15
2.11	Программа в файле lab8-3.asm	16
2.12	Запуск программы lab8-3.asm	16
2.13	Программа в файле task.asm	18
2.14	Запуск программы task.asm	19

Список таблиц

1 Цель работы

Целью работы является приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки..

2 Выполнение лабораторной работы

2.1 Реализация циклов в NASM

Создала каталог для программ лабораторной работы № 8 и файл lab8-1.asm.

При реализации циклов в NASM с использованием инструкции `loop` необходимо помнить, что эта инструкция использует регистр `ecx` в качестве счетчика, уменьшая его значение на единицу с каждым шагом. В качестве примера я рассмотрела программу, которая выводит текущее значение регистра `ecx`.

Добавила в файл lab8-1.asm текст программы из листинга 8.1. Затем создала исполняемый файл и проверила его работу.

```
duaaavahish@VirtualBox: ~/work/arch-...
/home/du~8-1.asm [----] 9 L:[ 1+27 28/ 28] *([*])[X]
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
mov [N],ecx
mov eax,[N]
call iprintLF ; Вывод значения `N`
loop label ; `ecx=ecx-1` и если `ecx` не `0`
; переход на `label`
call quit
```

Рисунок 2.1: Программа в файле lab8-1.asm

```
duaavahish@VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 lab8-1.o -o lab8-1
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 7
7
6
5
4
3
2
1
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.2: Запуск программы lab8-1.asm

Этот пример показал, что изменение регистра `ecx` внутри цикла `loop` может привести к некорректной работе программы. Я изменила текст программы, добавив модификацию регистра `ecx` в цикле.

Полученная программа: - Запускает бесконечный цикл при нечетном значении `N`. - Выводит только нечетные числа, если `N` четное.


```
duaaavahish@VirtualBox: ~/work/arch-...
/home/du~8-1.asm [----] 13 L:[ 1+25 26/ 30] *([*]
#include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
sub ecx,1 ; `ecx=ecx-1`
mov [N],ecx
mov eax,[N]
call iprintLf
loop label
; переход на `label`
call quit
```

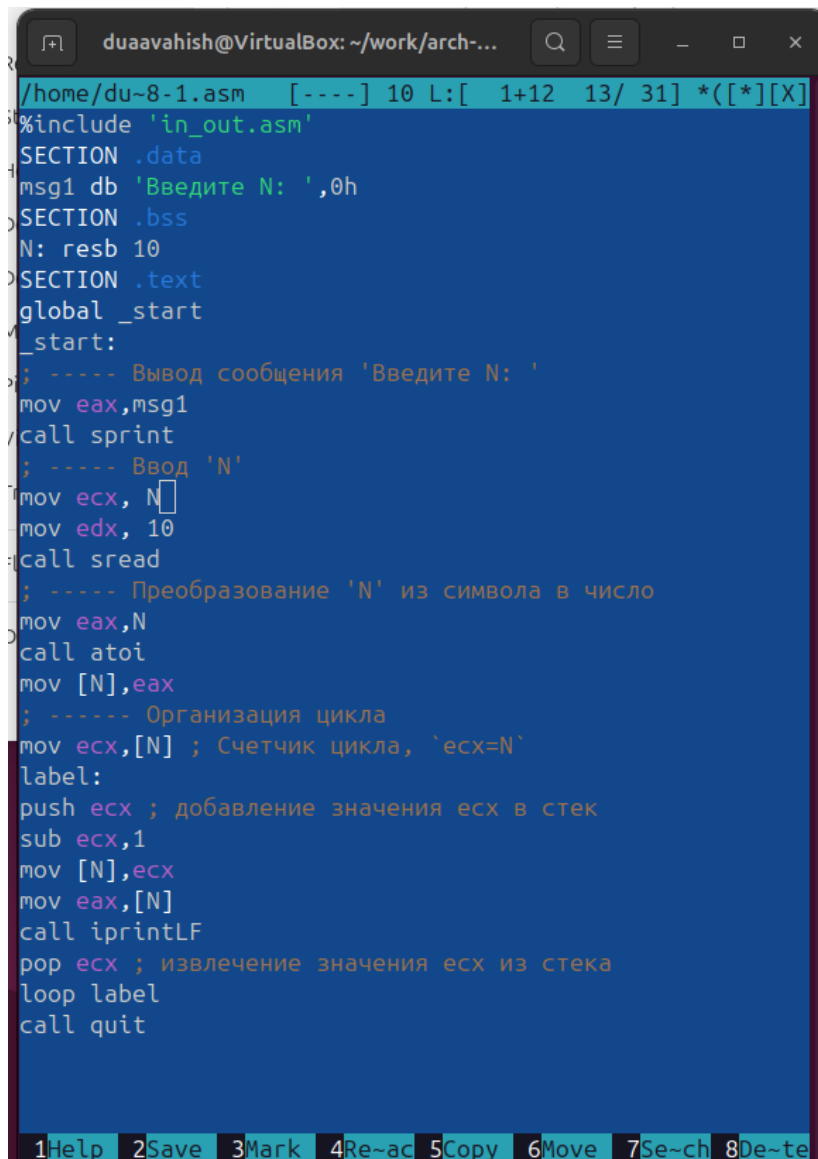
Рисунок 2.3: Программа в файле lab8-1.asm

```
4294771212
4294771210
4294771208
4294771206
4294771204
4294771202
4294771200
4294771198
4294771196
la 4294771194
4294771192
4294771190
4294771188
4294771186
p 42947711^C
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.4: Запуск программы lab8-1.asm

Для корректного использования регистра `ecx` в цикле, я добавила команды `push` и `pop`, чтобы временно сохранять значение регистра в стеке. Это позволило сохранить корректность работы программы. Внесла изменения в текст программы, создала исполняемый файл и проверила его работу.

Теперь программа: - Выводит числа от $N-1$ до 0 . - Число проходов цикла соответствует значению N .



```
/home/du~8-1.asm [----] 10 L:[ 1+12 13/ 31] *([*])[X]
%include 'in_out.asm'
SECTION .data
msg1 db 'Введите N: ',0h
SECTION .bss
N: resb 10
SECTION .text
global _start
_start:
; ----- Вывод сообщения 'Введите N: '
mov eax,msg1
call sprint
; ----- Ввод 'N'
mov ecx, N
mov edx, 10
call sread
; ----- Преобразование 'N' из символа в число
mov eax,N
call atoi
mov [N],eax
; ----- Организация цикла
mov ecx,[N] ; Счетчик цикла, `ecx=N`
label:
push ecx ; добавление значения ecx в стек
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx ; извлечение значения ecx из стека
loop label
call quit
```

1Help 2Save 3Mark 4Re~ac 5Copy 6Move 7Se~ch 8De~te

Рисунок 2.5: Программа в файле lab8-1.asm

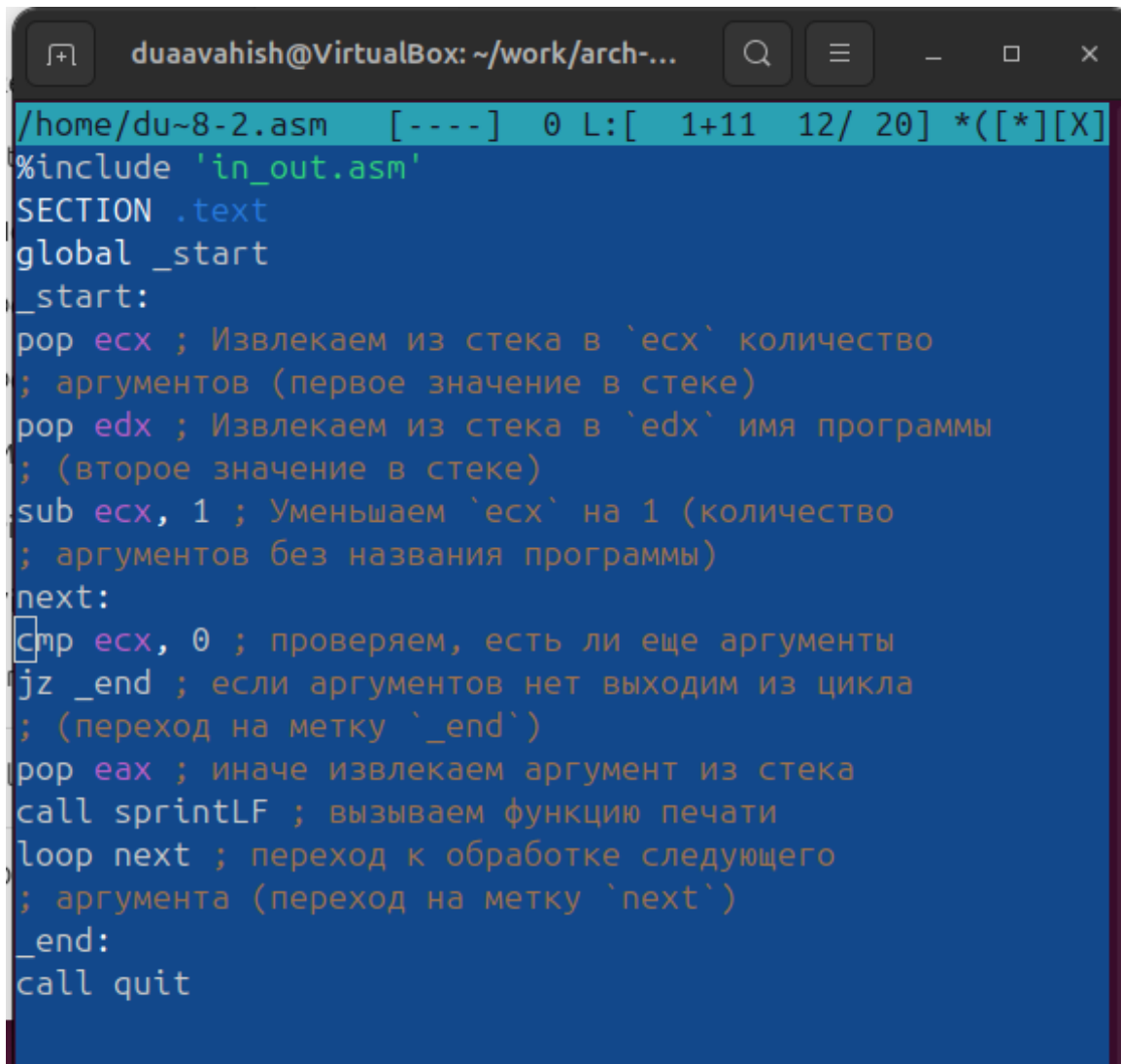
```
duaavahish@VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-1.asm
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 lab8-1.o -o lab8-1
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-1
Введите N: 7
6
5
4
3
2
1
0
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.6: Запуск программы lab8-1.asm

2.2 Обработка аргументов командной строки

Создала файл lab8-2.asm в каталоге ~/work/arch-pc/lab08 и ввела в него текст программы из листинга 8.2.

Создала исполняемый файл и запустила его, указав аргументы. Программа обработала 4 аргумента, разделенных пробелами (слова или числа).

A screenshot of a code editor window titled 'duaavahish@VirtualBox: ~/work/arch-...'. The editor shows assembly code for a file named 'lab8-2.asm'. The code includes a header line with file path and line numbers, followed by standard assembly directives and a loop that prints arguments from the stack. The code is as follows:

```
/home/duaavahish/work/arch-pc/lab08/lab8-2.asm [----] 0 L:[ 1+11 12/ 20] *([*])[X]
%include 'in_out.asm'
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
             ; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
             ; (второе значение в стеке)
    sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
             ; аргументов без названия программы)
next:
    cmp ecx, 0 ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
             ; (переход на метку `_end`)
    pop eax ; иначе извлекаем аргумент из стека
    call sprintf ; вызываем функцию печати
    loop next ; переход к обработке следующего
             ; аргумента (переход на метку `next`)
_end:
    call quit
```

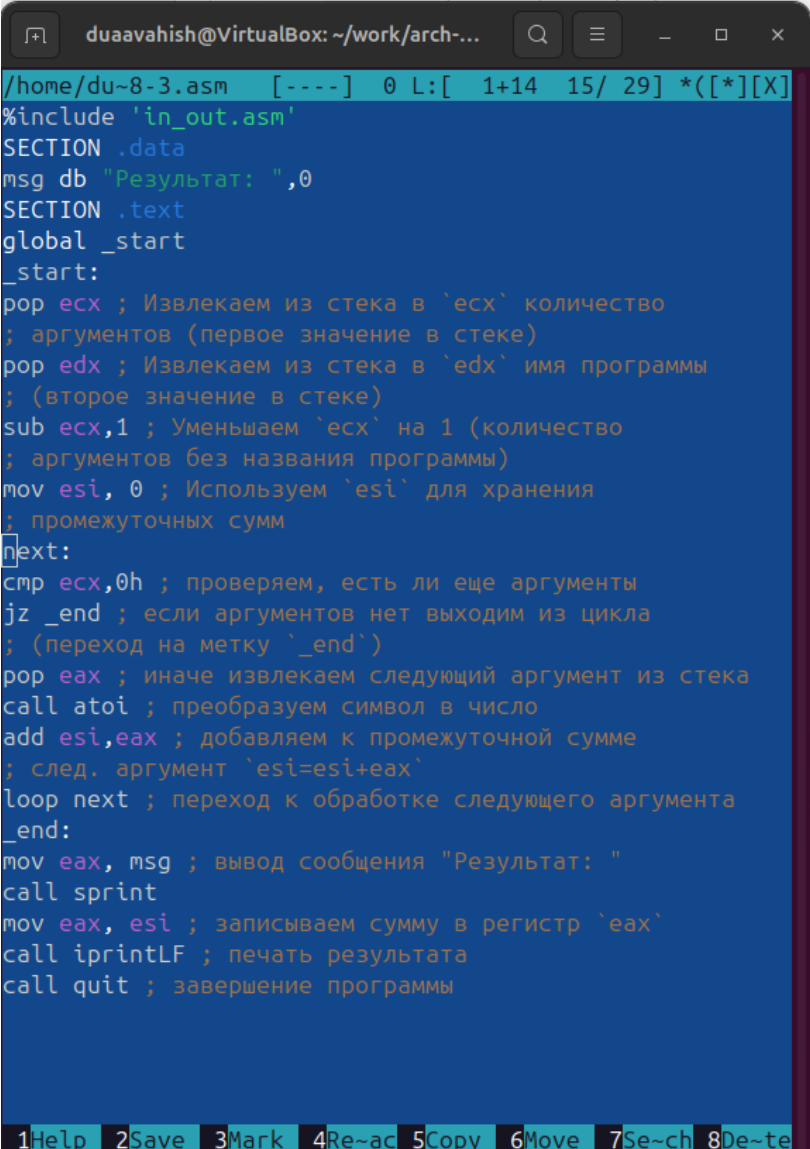
Рисунок 2.7: Программа в файле lab8-2.asm

A screenshot of a terminal window showing the compilation and execution of the assembly program. The user runs 'nasm -f elf lab8-2.asm' to compile the assembly code into an object file, then 'ld -m elf_i386 lab8-2.o -o lab8-2' to link it into an executable. Finally, they run './lab8-2' which prints the arguments passed to it. The terminal output is as follows:

```
duaavahish@VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-2.asm
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 lab8-2.o -o lab8-2
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-2
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-2 argument 1 argument 2 'argu
ment 3'
argument
1
argument
2
argument 3
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.8: Запуск программы lab8-2.asm

Рассмотрела пример программы, которая выводит сумму чисел, переданных в качестве аргументов командной строки.



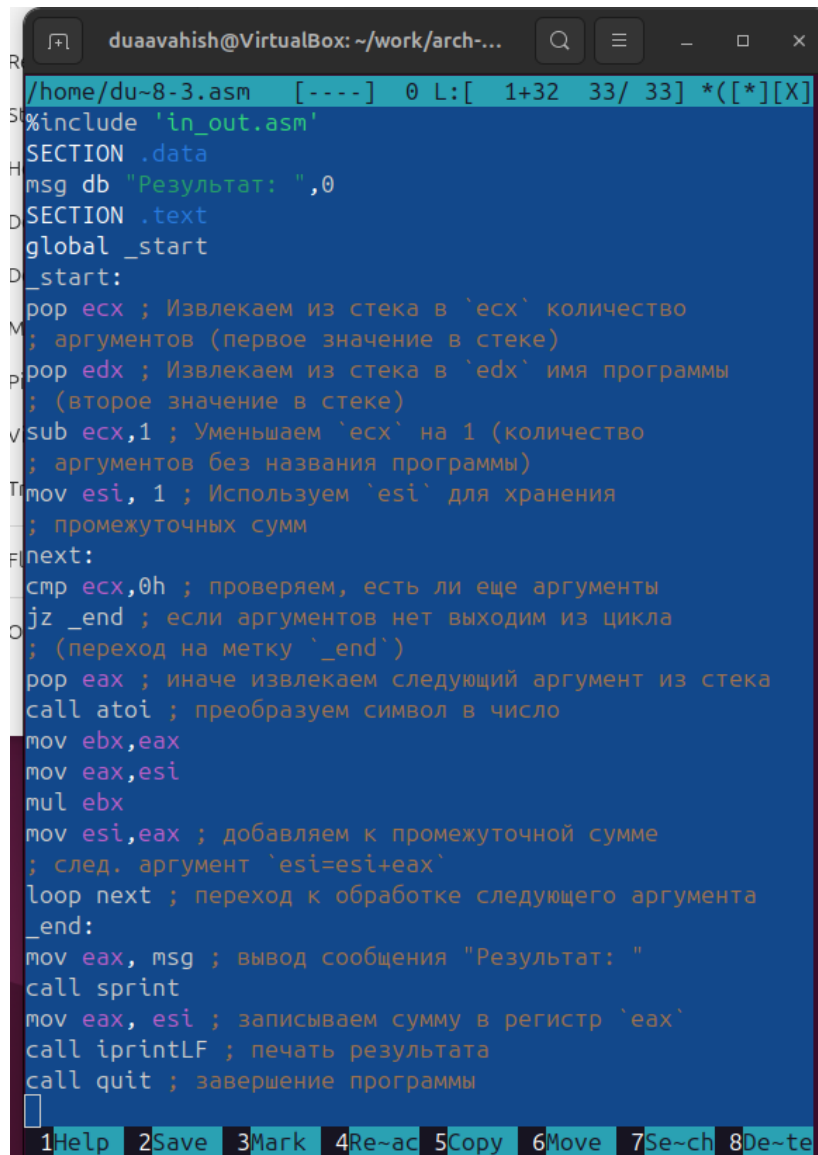
```
duaavahish@VirtualBox: ~/work/arch-...  
/home/du-8-3.asm [----] 0 L:[ 1+14 15/ 29] *([*][X]  
%include 'in_out.asm'  
SECTION .data  
msg db "Результат: ",0  
SECTION .text  
global _start  
_start:  
    pop ecx ; Извлекаем из стека в `ecx` количество  
            ; аргументов (первое значение в стеке)  
    pop edx ; Извлекаем из стека в `edx` имя программы  
            ; (второе значение в стеке)  
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество  
            ; аргументов без названия программы)  
    mov esi, 0 ; Используем `esi` для хранения  
            ; промежуточных сумм  
next:  
    cmp ecx,0h ; проверяем, есть ли еще аргументы  
    jz _end ; если аргументов нет выходим из цикла  
            ; (переход на метку `_end`)  
    pop eax ; иначе извлекаем следующий аргумент из стека  
    call atoi ; преобразуем символ в число  
    add esi,eax ; добавляем к промежуточной сумме  
            ; след. аргумент `esi=esi+eax`  
    loop next ; переход к обработке следующего аргумента  
_end:  
    mov eax, msg ; вывод сообщения "Результат: "  
    call sprint  
    mov eax, esi ; записываем сумму в регистр `eax`  
    call iprintLF ; печать результата  
    call quit ; завершение программы
```

Рисунок 2.9: Программа в файле lab8-3.asm

```
duaavahish@VirtualBox:~/work/arch-pc/lab08$  
duaavahish@VirtualBox:~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm  
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 lab8-3.o -o lab8-3  
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-3 2 3  
Результат: 5  
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./lab8-3 2 3 9 7 8 6 1 3  
Результат: 39  
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

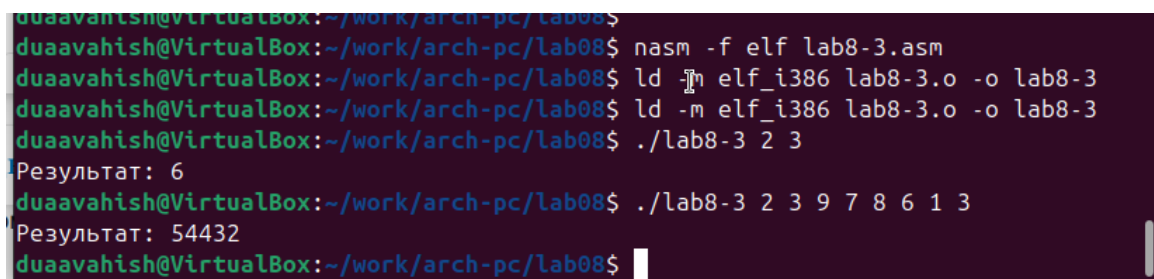
Рисунок 2.10: Запуск программы lab8-3.asm

Изменила текст программы из листинга 8.3, чтобы вычислять произведение аргументов.



```
/home/du~8-3.asm [---] 0 L: [ 1+32 33/ 33] *([*])[X]
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
SECTION .text
global _start
_start:
    pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
    pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
    sub ecx,1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
    mov esi, 1 ; Используем `esi` для хранения
; промежуточных сумм
next:
    cmp ecx,0h ; проверяем, есть ли еще аргументы
    jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
    pop eax ; иначе извлекаем следующий аргумент из стека
    call atoi ; преобразуем символ в число
    mov ebx,eax
    mov eax,esi
    mul ebx
    mov esi,eax ; добавляем к промежуточной сумме
; след. аргумент `esi=esi+eax`
    loop next ; переход к обработке следующего аргумента
_end:
    mov eax, msg ; вывод сообщения "Результат: "
    call sprint
    mov eax, esi ; записываем сумму в регистр `eax`
    call iprintLF ; печать результата
    call quit ; завершение программы
```

Рисунок 2.11: Программа в файле lab8-3.asm



```
duaavahish@VirtualBox: ~/work/arch-pc/lab08$ nasm -f elf lab8-3.asm
duaavahish@VirtualBox: ~/work/arch-pc/lab08$ ld -m elf_i386 lab8-3.o -o lab8-3
duaavahish@VirtualBox: ~/work/arch-pc/lab08$ ./lab8-3 2 3
Результат: 6
duaavahish@VirtualBox: ~/work/arch-pc/lab08$ ./lab8-3 2 3 9 7 8 6 1 3
Результат: 54432
duaavahish@VirtualBox: ~/work/arch-pc/lab08$
```

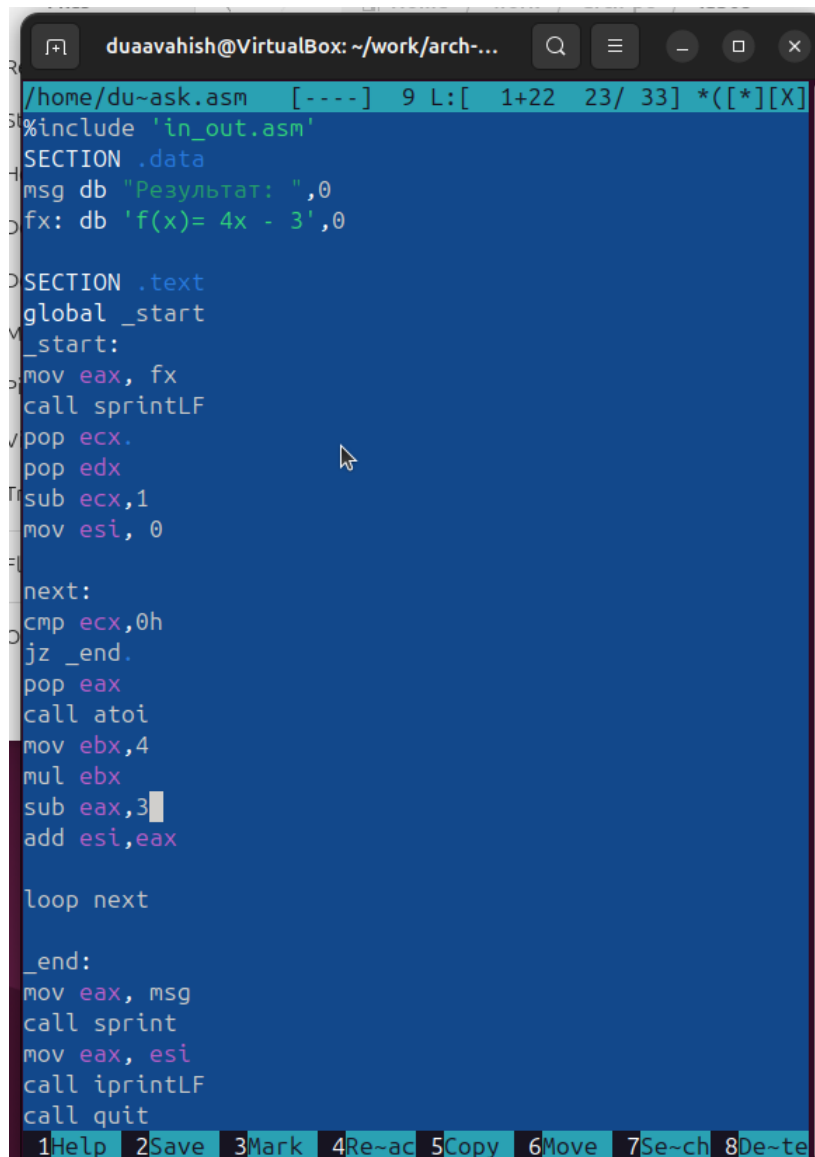
Рисунок 2.12: Запуск программы lab8-3.asm

2.3 Задание для самостоятельной работы

Написала программу для нахождения суммы значений функции $f(x)$ для $x = x_1, x_2, \dots, x_n$. Программа должна выводить значение $f(x_1) + f(x_2) + \dots + f(x_n)$.

Значения x передаются в программу как аргументы. Для функции $f(x)$ выбрала вариант 6: $f(x) = 4x - 3$.

Создала исполняемый файл и проверила его работу на нескольких наборах значений x .



```
/home/duaavahish@VirtualBox: ~/work/arch-...
/home/duaavahish@VirtualBox: ~/work/arch-... [----] 9 L: [ 1+22 23/ 33] *([*])[X]
%include 'in_out.asm'
SECTION .data
msg db "Результат: ",0
fx: db 'f(x)= 4x - 3',0

SECTION .text
global _start
_start:
mov eax, fx
call sprintf
pop ecx
pop edx
sub ecx,1
mov esi, 0

next:
cmp ecx,0h
jz _end
pop eax
call atoi
mov ebx,4
mul ebx
sub eax,3
add esi,eax

loop next

_end:
mov eax, msg
call sprintf
mov eax, esi
call iprintLF
call quit
```

Рисунок 2.13: Программа в файле task.asm

Для проверки запустила программу сначала с одним аргументом: - При $x = 1$, $f(1) = 1$. - При $x = 2$, $f(2) = 5$.

Затем передала несколько аргументов и получила сумму значений функции.

```
duaavahish@VirtualBox:~/work/arch-pc/lab08$ nasm -f elf task.asm
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ld -m elf_i386 task.o -o task
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./task
f(x)= 4x - 3
Результат: 0
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./task 1
f(x)= 4x - 3
Результат: 1
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./task 2
f(x)= 4x - 3
Результат: 5
duaavahish@VirtualBox:~/work/arch-pc/lab08$ ./task 2 3 4 6 7 9
f(x)= 4x - 3
Результат: 106
duaavahish@VirtualBox:~/work/arch-pc/lab08$
```

Рисунок 2.14: Запуск программы task.asm

3 Выводы

Освоили работы со стеком, циклом и аргументами на ассемблере `nasm`.